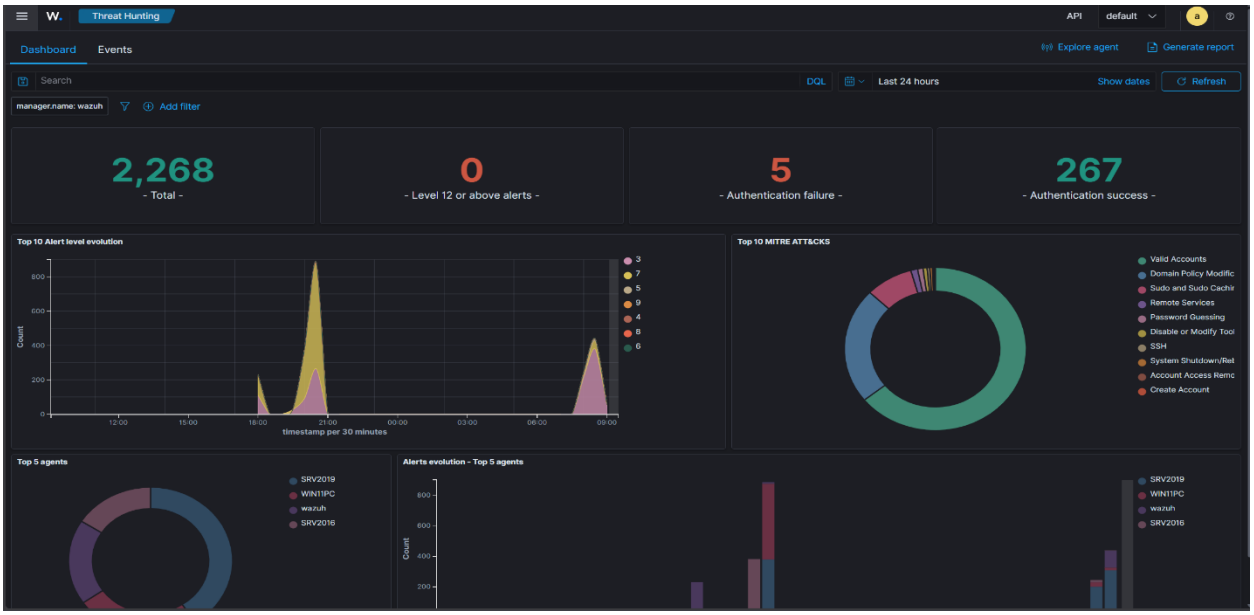
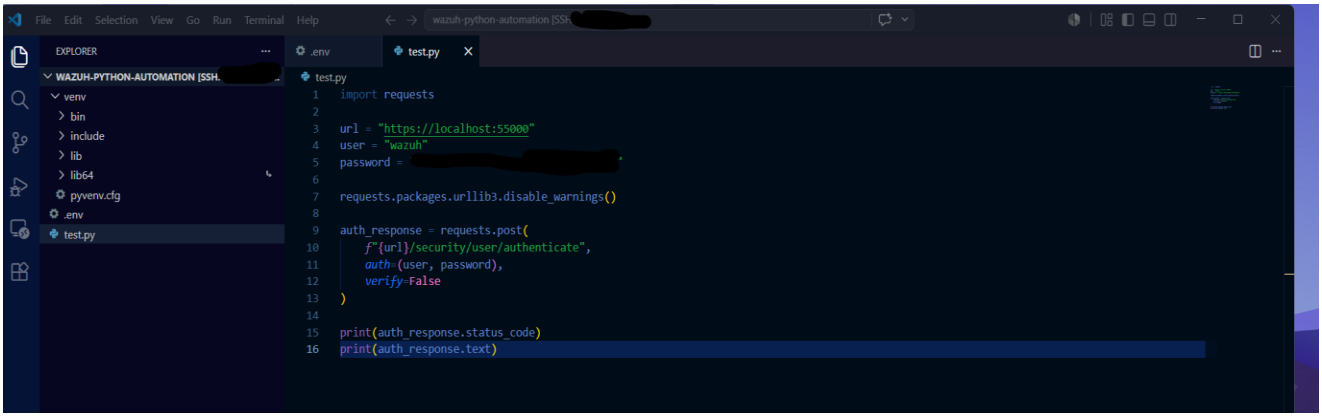
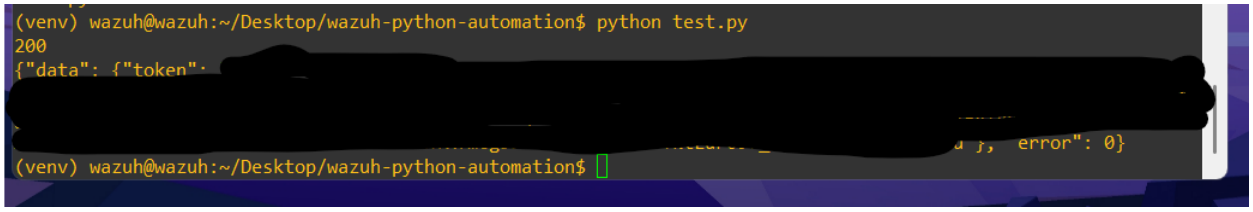
## Wazuh Python Automation with MCP



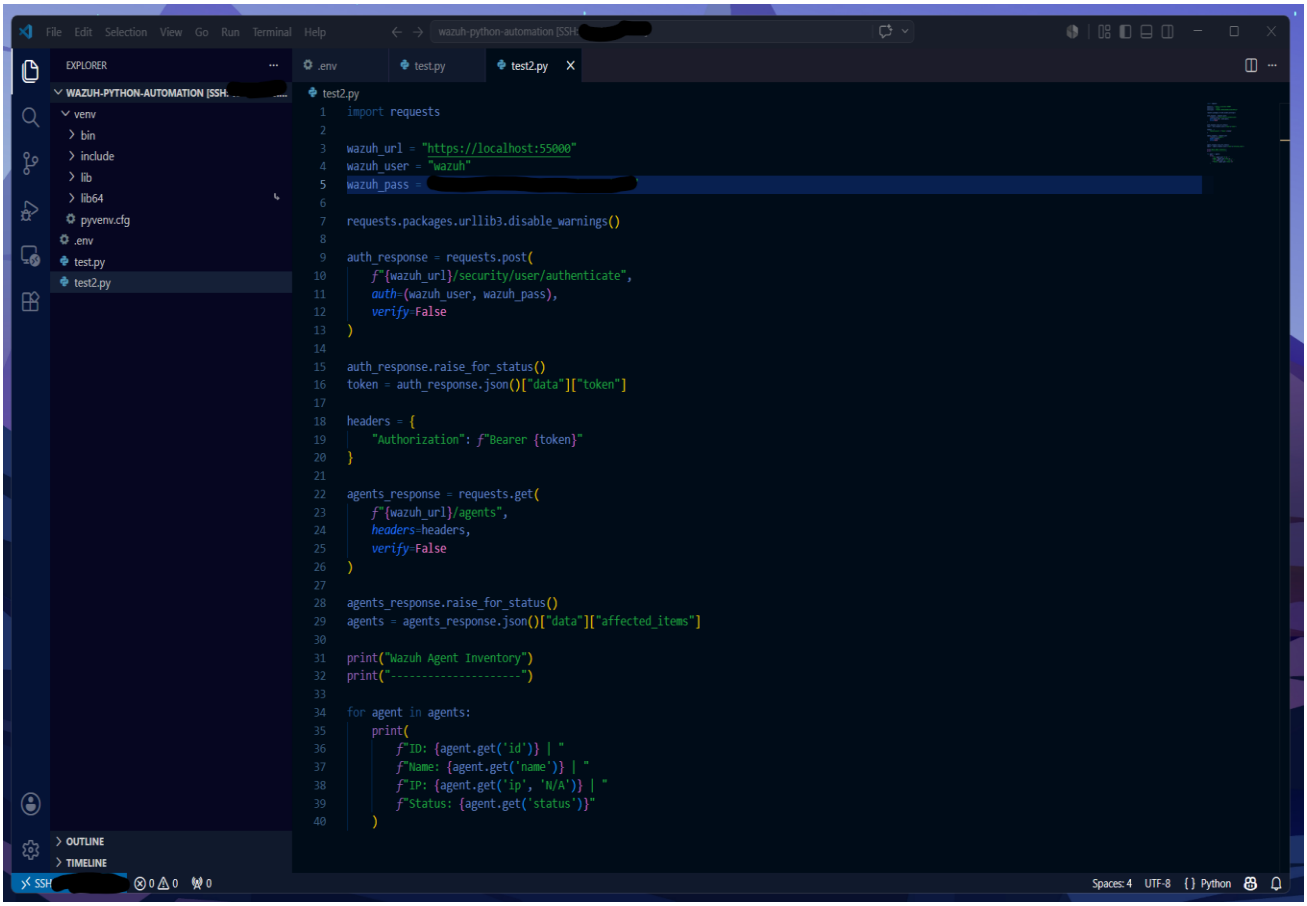

This project covers the deployment of a Wazuh SIEM, the development of Python-based automation for security operations tasks, and the integration of Model Context Protocol (MCP) to enable AI-assisted interaction with security data. The solution provides automated agent inventory management, endpoint health monitoring, alert analysis, SOC reporting, and natural language access to security information through custom MCP tools.

### Part 1: Python Automation



The first task I did was configure SSH access into my ubuntu VM where the wazuh server is hosted in VS code, so I can edit my code easier. I first created a test python script for basic wazuh API authentication. Then in my SSH session in PuTTY into my VM, I typed **python test.py,** with it returning a 200 code, with the token blurred out, confirming it works.



```
import requests

wazuh_url = "https://localhost:55000"
wazuh_user = "wazuh"
wazuh_pass = [redacted]

requests.packages.urllib3.disable_warnings()

auth_response = requests.post(
    f"{wazuh_url}/security/user/authenticate",
    auth=(wazuh_user, wazuh_pass),
    verify=False
)

auth_response.raise_for_status()
token = auth_response.json()["data"]["token"]

headers = {
    "Authorization": f"Bearer {token}"
}

agents_response = requests.get(
    f"{wazuh_url}/agents",
    headers=headers,
    verify=False
)

agents_response.raise_for_status()
agents = agents_response.json()["data"]["affected_items"]

print("Wazuh Agent Inventory")
print("---------------------")

for agent in agents:
    print(
        f"ID: {agent.get('id')} | "
        f"Name: {agent.get('name')} | "
        f"IP: {agent.get('ip', 'N/A')} | "
        f"Status: {agent.get('status')}"
    )
```

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python test2.py
Wazuh Agent Inventory
---------------------
ID: 000 | Name: wazuh | IP: 127.0.0.1 | Status: active
ID: 002 | Name: SRV2016 | IP: [redacted] | Status: active
ID: 003 | Name: SRV2019 | IP: [redacted] | Status: active
ID: 004 | Name: WIN11PC | IP: [redacted] | Status: pending
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

Next, I created an automated reporting function that analyzes agent status information and identifies systems that are no longer reporting as active. This functionality assists security administrators in quickly identifying disconnected, pending, or potentially unhealthy endpoints requiring investigation. Back to the SSH session, running the script with **python test2** confirms it is working.

```python
import requests

wazuh_url = "https://localhost:55000"
wazuh_user = "wazuh"
wazuh_pass = "[redacted]"

requests.packages.urllib3.disable_warnings()

auth_response = requests.post(
    f"{wazuh_url}/security/user/authenticate",
    auth=(wazuh_user, wazuh_pass),
    verify=False
)

auth_response.raise_for_status()
token = auth_response.json()["data"]["token"]

headers = {
    "Authorization": f"Bearer {token}"
}

agents_response = requests.get(
    f"{wazuh_url}/agents",
    headers=headers,
    verify=False
)

agents_response.raise_for_status()
agents = agents_response.json()["data"]["affected_items"]

print("\nOffline Agent Report")
print("=" * 50)

offline_found = False

for agent in agents:
    status = agent.get("status", "").lower()

    if status != "active":
        offline_found = True

        print(
            f"ID: {agent.get('id')} | "
            f"Name: {agent.get('name')} | "
            f"IP: {agent.get('ip', 'N/A')} | "
            f"Status: {agent.get('status')}"
        )

if not offline_found:
    print("All agents are active.")
```

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python offline-agents.py

Offline Agent Report
==================================================
ID: 004 | Name: WIN11PC | IP: [redacted] | Status: pending
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

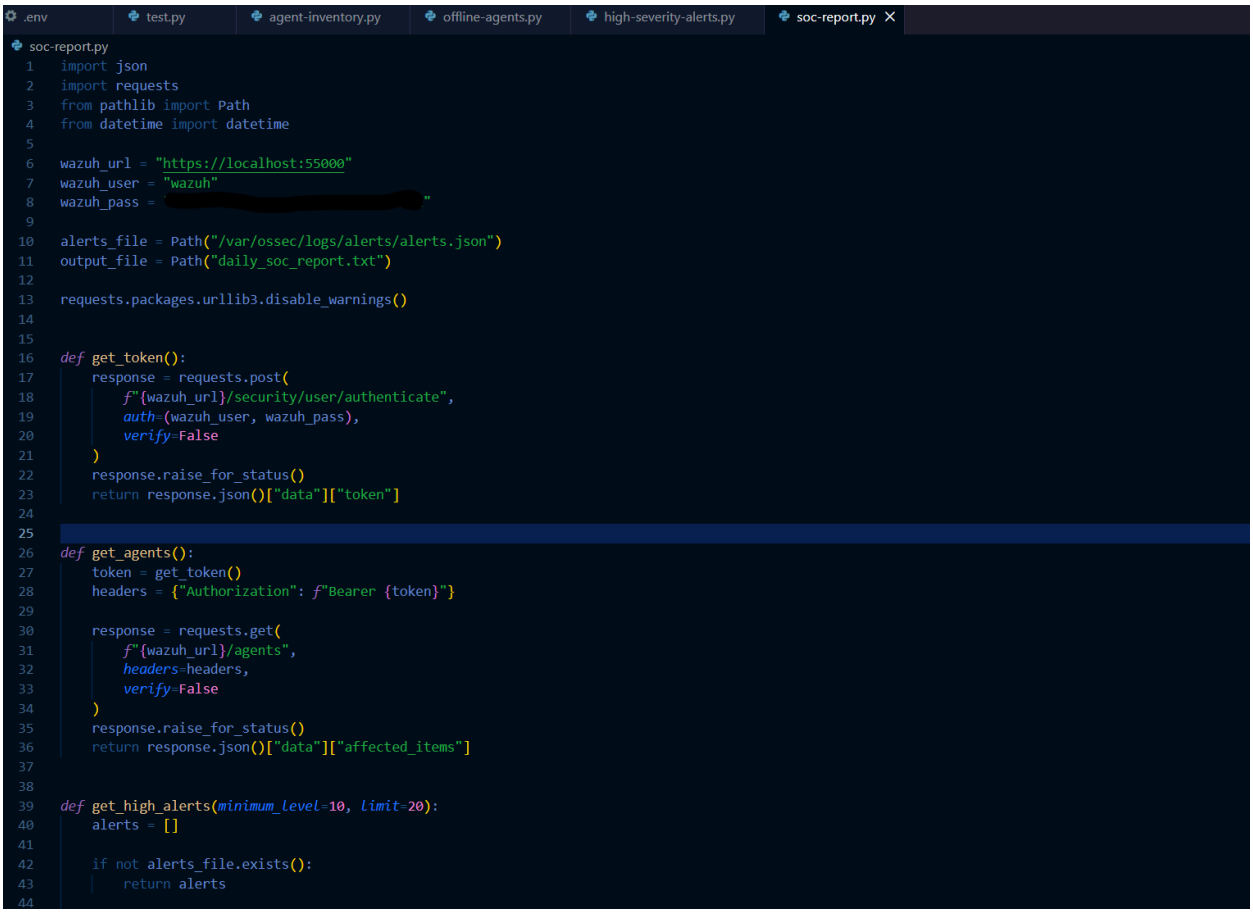
Next, I developed an automated Security Operations Center (SOC) reporting script that consolidates agent health information, offline endpoint status, alert activity, and security findings into a single daily report.

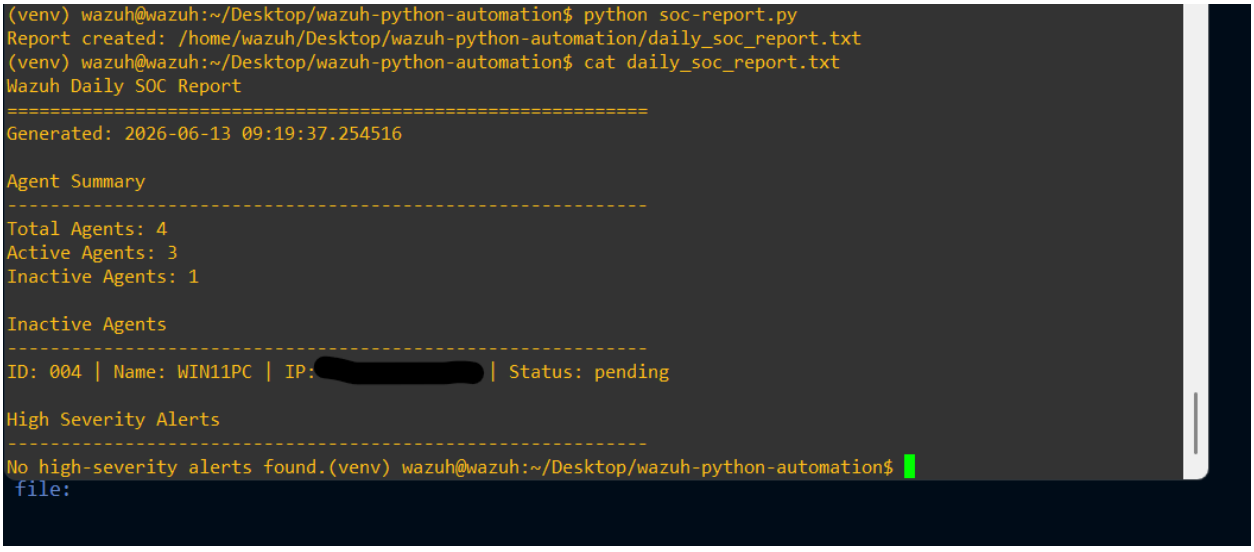
high-severity-alerts.py

```python
import json
from pathlib import Path

alerts_file = Path("/var/ossec/logs/alerts/alerts.json")
minimum_level = 10
limit = 20

print("\nHigh Severity Alerts")
print("=" * 70)

if not alerts_file.exists():
    print(f"Alerts file not found: {alerts_file}")
    raise SystemExit(1)

matches = []

with alerts_file.open("r", encoding="utf-8", errors="ignore") as file:
    for line in file:
        try:
            alert = json.loads(line)
        except json.JSONDecodeError:
            continue

        rule = alert.get("rule", {})
        level = rule.get("level", 0)

        if level >= minimum_level:
            matches.append(alert)

for alert in matches[-limit:]:
    rule = alert.get("rule", {})
    agent = alert.get("agent", {})

    print(f"Time: {alert.get('timestamp')}")
    print(f"Agent: {agent.get('name', 'N/A')}")
    print(f"Agent ID: {agent.get('id', 'N/A')}")
    print(f"Rule ID: {rule.get('id')}")
    print(f"Level: {rule.get('level')}")
    print(f"Description: {rule.get('description')}")
    print("-" * 70)

if not matches:
    print("No high-severity alerts found.")
```

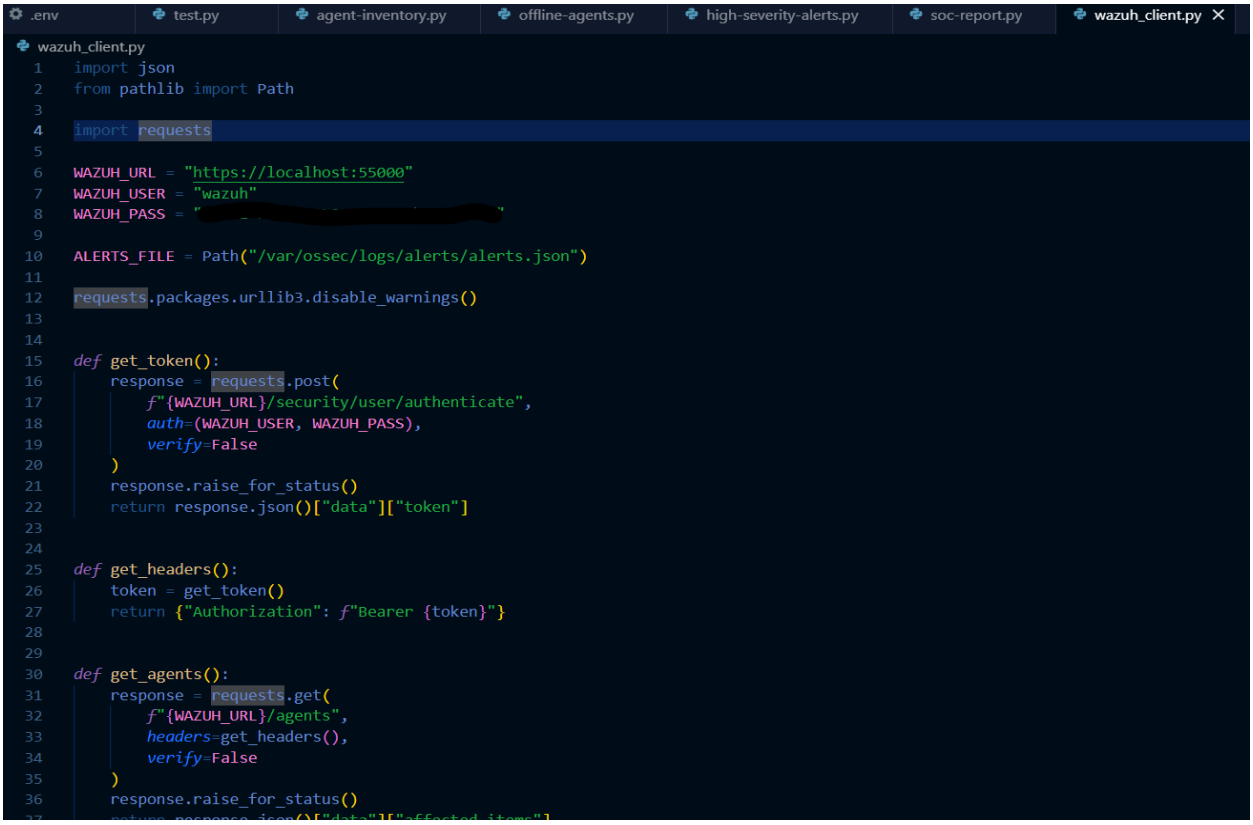
```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python high-severity-alerts.py

High Severity Alerts
======================================================================
No high-severity alerts found.
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

Next, I configured **High-Severity Alert Monitoring:** Created a Python script to identify and report high-severity alerts from Wazuh alert logs.




The following images demonstrate the testing of a **Daily SOC Report Generation:** a Python script that authenticates to the Wazuh API, collects agent and alert data, and generates a consolidated daily SOC report for security monitoring.




I then proceeded further with the wazuh client module development, a centralized Python module to handle Wazuh API authentication, agent retrieval, alert processing, and shared functionality used throughout the automation project.


A test script was then made to validate agent inventory retrieval, offline agent detection, and high-severity alert functions from the centralized Wazuh client module.

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python test_client.py
Agents:
000 wazuh active
002 SRV2016 active
003 SRV2019 active
004 WIN11PC pending

Offline Agents:
004 WIN11PC pending

High Severity Alerts:
No high-severity alerts found.
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$
```

I then ran the test_client script to ensure full functionality of it.

**Part 2: MCP**

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ pip install mcp
Collecting mcp
  Downloading mcp-1.27.2-py3-none-any.whl (220 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 220.5/220.5 KB 5.0 MB/s eta 0:00:00
Collecting pyjwt[crypto]>=2.10.1
  Downloading pyjwt-2.13.0-py3-none-any.whl (31 kB)
Collecting typing-extensions>=4.9.0
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.6/44.6 KB 20.8 MB/s eta 0:00:00
Collecting python-multipart>=0.0.9
  Downloading python_multipart-0.0.32-py3-none-any.whl (30 kB)
Collecting pydantic<3.0.0,>=2.11.0
  Downloading pydantic-2.13.4-py3-none-any.whl (472 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 472.3/472.3 KB 23.2 MB/s eta 0:00:00
Collecting starlette>=0.27
  Downloading starlette-1.3.1-py3-none-any.whl (73 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 73.6/73.6 KB 30.2 MB/s eta 0:00:00
Collecting anyio>=4.5
  Downloading anyio-4.13.0-py3-none-any.whl (114 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 114.4/114.4 KB 53.6 MB/s eta 0:00:00
Collecting pydantic-settings>=2.5.2
  Downloading pydantic_settings-2.14.1-py3-none-any.whl (60 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 61.0/61.0 KB 26.7 MB/s eta 0:00:00
Collecting httpx<1.0.0,>=0.27.1
```

Next, I ran the command **pip install mcp** to install the MCP server on the ubuntu vm.

mcp_server.py

```python
from mcp.server.fastmcp import FastMCP

from wazuh_client import (
    get_agents,
    get_offline_agents,
    get_high_severity_alerts,
)

mcp = FastMCP("wazuh-soc-mcp")


@mcp.tool()
def list_agents() -> list:
    """List all Wazuh agents with ID, name, IP, and status."""
    agents = get_agents()

    return [
        {
            "id": agent.get("id"),
            "name": agent.get("name"),
            "ip": agent.get("ip", "N/A"),
            "status": agent.get("status"),
        }
        for agent in agents
    ]


@mcp.tool()
def list_offline_agents() -> list:
    """List Wazuh agents that are not active."""
    agents = get_offline_agents()

    return [
        {
            "id": agent.get("id"),
            "name": agent.get("name"),
            "ip": agent.get("ip", "N/A"),
            "status": agent.get("status"),
        }
        for agent in agents
    ]
```

```
(venv) wazuh@wazuh:~/Desktop/wazuh-python-automation$ python3 mcp_server.py
```

When initially launched, the MCP server produced no console output, which is expected behavior because it waits for incoming tool requests.

mcp_server.py

```python
@mcp.tool()
def list_high_severity_alerts(minimum_level: int = 10, limit: int = 10) -> list:
    """List recent high-severity Wazuh alerts."""
    alerts = get_high_severity_alerts(
        minimum_level=minimum_level,
        limit=limit
    )

    results = []

    for alert in alerts:
        rule = alert.get("rule", {})
        agent = alert.get("agent", {})

        results.append(
            {
                "timestamp": alert.get("timestamp"),
                "agent": agent.get("name", "N/A"),
                "agent_id": agent.get("id", "N/A"),
                "rule_id": rule.get("id"),
                "level": rule.get("level"),
                "description": rule.get("description"),
            }
        )

    return results

def get_agent_health_summary():
    agents = get_agents()

    total_agents = len(agents)

    active_agents = [
        a for a in agents
        if a.get("status", "").lower() == "active"
    ]

    offline_agents = [
        a for a in agents
        if a.get("status", "").lower() != "active"
    ]

    return {
        "total_agents": total_agents,
        "active_agents": len(active_agents),
        "offline_agents": len(offline_agents),
        "offline_agent_names": [
            agent.get("name")
            for agent in offline_agents
        ]
    }
```

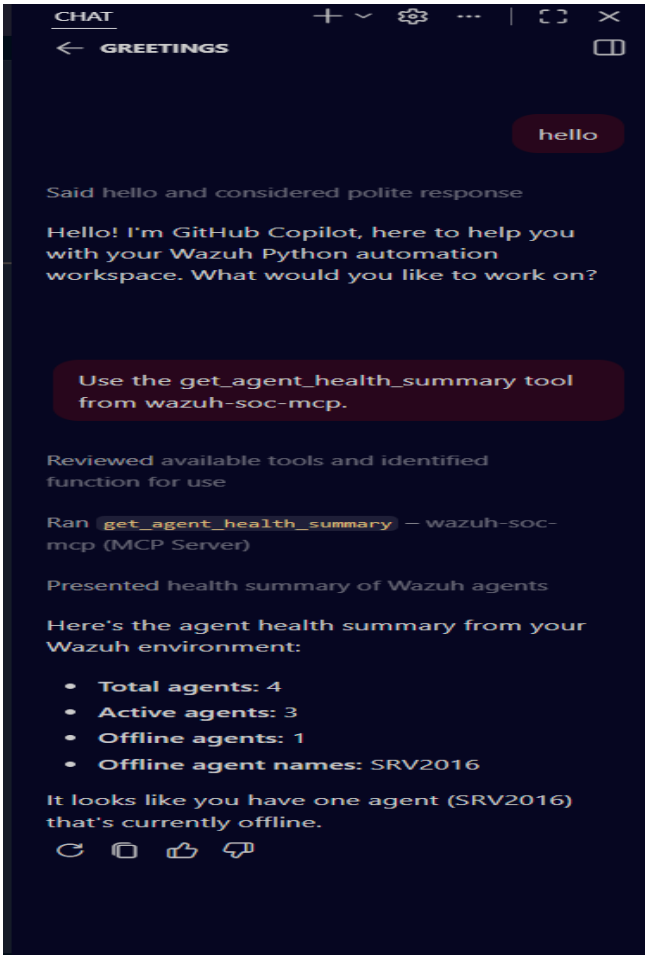
This is part of the FastMCP server script with a few more functions that once I got the MCP server running, I wanted it to test.

mcp_server.py  {} mcp.json

.vscode > {} mcp.json > ...

```
{
  "servers": {
    Start | 4 tools | More...
    "wazuh-soc-mcp": {
      "type": "stdio",
      "command": "/home/wazuh/Desktop/wazuh-python-automation/venv/bin/python",
      "args": [
        "/home/wazuh/Desktop/wazuh-python-automation/mcp_server.py"
      ]
    }
  },
  "inputs": []
}
```

Next, I created an mcp.json file I can go to for when I want to start running the MCP server defining the server runtime, Python environment, and execution path in the mcp.json configuration file.



Next in VScode, I typed ctrl + shift + p, and typed Chat: Open Chat, which opened this sidebar here of things I can ask or tell it to do. This shows an end-to-end workflow that combines Wazuh monitoring, Python automation, MCP integration, and AI-assisted security operations. The resulting solution allows security information to be queried, summarized, and reported through natural language interactions while maintaining direct integration with the underlying Wazuh platform.


This displays lastly the development of a Wazuh-powered security monitoring and automation platform that combines SIEM functionality, Python-based operational tooling, and MCP-enabled AI integration. The project demonstrates practical application of cybersecurity automation, endpoint monitoring, alert analysis, SOC reporting, and AI-assisted security operations workflows. This project demonstrates how traditional SIEM monitoring can be enhanced through Python automation and AI integration, reducing manual effort while improving accessibility to security information.